SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO4: Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)

Assessment Tool Weightage: 40%

Duration : 45 Minutes
QUESTIONS: 2 Total Marks:20

Annexure A

INTEGRATED EXPERIMENTS

Code of Conduct:

I Nalluri Akhilesh Kumar(192365074) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Nalluri Akhilesh Kumar
Signature of the student:

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

(a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.
(b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification
(c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

(a) Dataset Loading, Preprocessing and Splitting

Dataset Description

The Iris dataset contains 150 flower samples belonging to three classes:

- Setosa
- Versicolor
- Virginica

It contains four numerical features:

1. Sepal Length
2. Sepal Width
3. Petal Length
4. Petal Width

There are no missing values in the standard Iris dataset, so no imputation is required. Since all four input features are numerical, categorical encoding is also unnecessary.

Python Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
import pandas as pd
import matplotlib.pyplot as plt

# Load dataset
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df["target"] = iris.target

# Display first 5 rows
print("First 5 rows:")
print(df.head())

# Display data types
```

```
print("\nData types:")
print(df.dtypes)

# Check missing values
print("\nMissing values:")
print(df.isnull().sum())

# Separate features and target
X = df.drop("target", axis=1)
y = df["target"]

# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.30,
    random_state=42,
    stratify=y
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))
```

Expected First Five Rows

Sepal Length	Sepal Width	Petal Length	Petal Width	Target
5.1	3.5	1.4	0.2	0
4.9	3.0	1.4	0.2	0
4.7	3.2	1.3	0.2	0
4.6	3.1	1.5	0.2	0
5.0	3.6	1.4	0.2	0

Data Types

The four feature columns are generally:

`float64`

and the target is:

`int64`

Preprocessing Analysis

The dataset requires minimal preprocessing because:

- There are no missing values.
- All predictor variables are numerical.
- No categorical encoding is required.
- Decision Trees do not require feature normalization or standardization.

A 70:30 stratified split is used so that all three flower classes are represented proportionally in both training and testing data.

(b) Building the Decision Tree

The Decision Tree is built using the Gini Index as the splitting criterion.

Python Code

```
# Create Decision Tree classifier
model = DecisionTreeClassifier(
    criterion="gini",
    random_state=42
)

# Train the model
model.fit(X_train, y_train)

# Print tree structure
tree_rules = export_text(
    model,
    feature_names=list(X.columns)
)

print("\nDecision Tree Structure:")
print(tree_rules)

# Identify root node
root_index = model.tree_.feature[0]
root_feature = X.columns[root_index]

print("\nRoot Node:", root_feature)

# Plot the tree
plt.figure(figsize=(18, 10))

plot_tree(
    model,
    feature_names=X.columns,
    class_names=iris.target_names,
    filled=True
)

plt.title("Decision Tree - Iris Dataset")
plt.show()
```

Root Node

For the standard Iris dataset, the Decision Tree generally selects:

Petal Length (cm)

or, depending on the exact train/test split and tree configuration, a closely related petal feature can appear as the first split.

The root is selected because it provides the greatest reduction in impurity among the available features.

Gini Index

The Gini impurity is calculated as:

where p_i represents the proportion of samples belonging to class i.

A good split produces child nodes with lower impurity.

Therefore:

The root feature is selected because its split produces the largest reduction in Gini impurity and creates purer child nodes.

Interpretation

Petal-related measurements are highly discriminative in the Iris dataset. In particular, Setosa flowers are substantially different from Versicolor and Virginica in their petal dimensions.

Therefore, a petal feature is expected to become one of the earliest and most important decision rules.

(c) Model Evaluation

Python Code

```
# Make predictions
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("\nAccuracy:", accuracy)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# Classification Report
print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=iris.target_names
    )
)
```

Typical Result

With the specified `random_state=42`, the model should achieve approximately:

Accuracy ≈ 0.93

or approximately:

93%

The exact result can vary depending on the tree configuration and split.

Confusion Matrix

A typical confusion matrix may look like:

```
[[15  0  0]
 [ 0 14  1]
 [ 0  2 13]]
```

This means:

- 15 Setosa correctly classified.
- 14 Versicolor correctly classified.
- 13 Virginica correctly classified.
- A small number of Versicolor and Virginica samples were confused.

Classification Report

The classification report provides:

- Precision
- Recall
- F1-score
- Support

For example:

```
              precision    recall    f1-score

setosa          1.00        1.00       1.00
versicolor      0.88        0.93       0.90
virginica       0.93        0.87       0.90
```

The exact values depend on the generated train-test split.

Misclassified Instances

The following code identifies the actual misclassified samples rather than assuming them:

```
# Find misclassified instances
misclassified = X_test[y_test != y_pred].copy()

misclassified["Actual"] = y_test[y_test != y_pred]
misclassified["Predicted"] = y_pred[y_test != y_pred]

print("\nMisclassified Instances:")
print(misclassified)
```

For example, the model may confuse:

```
Versicolor → Virginica
Virginica → Versicolor
```

These errors are understandable because Versicolor and Virginica have overlapping petal measurements.

Performance Analysis

The Decision Tree performs well because:

1. The Iris dataset is relatively small and well structured.
2. Petal measurements strongly distinguish the three classes.
3. Decision Trees can learn nonlinear decision boundaries.
4. The model does not require feature scaling.
5. The resulting rules are easy to interpret.

However, the model can overfit if the tree is allowed to grow without restrictions.

Conclusion – Experiment 1

The Decision Tree successfully classified Iris flowers with high accuracy. Petal-related features were highly informative and therefore appeared near the root of the tree. The confusion matrix showed that most samples were correctly classified, while the small number of errors occurred mainly between Versicolor and Virginica. This experiment demonstrates how Gini impurity, recursive partitioning, tree interpretation, and classification metrics work together in a practical machine learning system.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

(a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, −1 each step, −5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α, γ, ε).

(b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.

(c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

Objective

To implement Q-Learning for a 4×4 Grid World environment, train the agent for at least 100 episodes, observe the evolution of Q-values, extract the optimal policy, and analyze cumulative reward convergence.

(a) Environment Definition

We create a 4×4 Grid World.

Example environment:

```
+----+----+----+----+
| S  |    |    |    |
+----+----+----+----+
|    | X  |    |    |
+----+----+----+----+
|    |    | X  |    |
+----+----+----+----+
|    |    |    | G  |
+----+----+----+----+
```

Where:

- `S` = Starting state
- `G` = Goal
- `X` = Obstacle
- Empty cells = normal states

States

There are:

4 × 4 = 16 states

They can be numbered:

```
0   1   2   3
4   5   6   7
8   9   10  11
12  13  14  15
```

Suppose:

- Start = State 0
- Goal = State 15
- Obstacles = States 5 and 10

Actions

Four actions are available:

```
0 = Up
1 = Down
2 = Left
3 = Right
```

Reward Structure

Situation	Reward
Reach Goal	+10
Normal step	-1
Hit obstacle	-5

Hyperparameters

We use:

```
Learning Rate (α) = 0.1
Discount Factor (γ) = 0.9
Exploration Rate (ε) = 1.0
Minimum ε = 0.01
Epsilon Decay = 0.995
Episodes = 100
```

Q-Table

The Q-table has:

16 states × 4 actions = 64 Q-values

Initially:

```
Q(s,a) = 0
```

for every state-action pair.

Complete Python Implementation

```
import numpy as np
import matplotlib.pyplot as plt

# Grid size
ROWS = 4
COLS = 4
N_STATES = ROWS * COLS
N_ACTIONS = 4

# Actions
UP = 0
DOWN = 1
LEFT = 2
RIGHT = 3

actions = ["U", "D", "L", "R"]
```

```
# Start and goal
START = 0
GOAL = 15

# Obstacles
OBSTACLES = {5, 10}

# Hyperparameters
alpha = 0.1
gamma = 0.9
epsilon = 1.0
epsilon_min = 0.01
epsilon_decay = 0.995

episodes = 100

# Initialize Q-table
Q = np.zeros((N_STATES, N_ACTIONS))


# Convert state number to row and column
def state_to_position(state):
    return divmod(state, COLS)


# Convert row and column to state number
def position_to_state(row, col):
    return row * COLS + col


# Environment step function
def step(state, action):

    row, col = state_to_position(state)

    new_row = row
    new_col = col

    if action == UP:
        new_row -= 1

    elif action == DOWN:
        new_row += 1

    elif action == LEFT:
        new_col -= 1

    elif action == RIGHT:
        new_col += 1

    # Boundary checking
    if new_row < 0 or new_row >= ROWS or new_col < 0 or new_col >= COLS:
        return state, -5, False

    new_state = position_to_state(new_row, new_col)

    # Obstacle
    if new_state in OBSTACLES:
        return state, -5, False
```

```
    # Goal
    if new_state == GOAL:
        return new_state, 10, True

    # Normal movement
    return new_state, -1, False


# Choose action using epsilon-greedy strategy
def choose_action(state, epsilon):

    if np.random.random() < epsilon:
        return np.random.randint(N_ACTIONS)

    return np.argmax(Q[state])


# Store cumulative rewards
episode_rewards = []

# Store Q-values
q_history = {
    1: None,
    50: None,
    100: None
}


# Q-Learning
for episode in range(1, episodes + 1):

    state = START
    total_reward = 0

    for step_count in range(100):

        # Choose action
        action = choose_action(state, epsilon)

        # Take action
        next_state, reward, done = step(state, action)

        # Q-Learning update
        old_value = Q[state, action]

        best_next_value = np.max(Q[next_state])

        Q[state, action] = old_value + alpha * (
            reward + gamma * best_next_value - old_value
        )

        # Update state
        state = next_state

        total_reward += reward

        if done:
            break

    episode_rewards.append(total_reward)
```

```
    # Save Q-table at selected episodes
    if episode in q_history:
        q_history[episode] = Q.copy()

    # Decay epsilon
    epsilon = max(
        epsilon_min,
        epsilon * epsilon_decay
    )


# Display Q-values for representative states
representative_states = [0, 1]

for episode in [1, 50, 100]:

    print("\nEpisode:", episode)

    for state in representative_states:

        print(
            "State",
            state,
            "Q-values:",
            np.round(q_history[episode][state], 3)
        )


# Display final Q-table
print("\nFinal Q-Table:")
print(np.round(Q, 3))
```

(b) Q-Table Evolution

Two representative states can be selected:

- State 0: Starting state
- State 1: State immediately to the right of the starting state

The program records their Q-values at Episodes:

- 1
- 50
- 100

An example of the expected trend is:

Episode	State	Up		Down	Left	Right
1	0	0		-0.1	-0.5	0
1	1	-0.1		-0.1	-0.1	0
50	0	-0.5		4.2	-0.5	5.1
50	1	-0.4		5.0	-0.3	6.2
100	0	-0.6		6.8	-0.6	7.4
100	1	-0.5		7.2	-0.4	8.1

Important: The exact numerical values will vary because Q-Learning uses random exploration. The values generated by running the code should be used as the final experimental values.

Q-Learning Update

The fundamental update equation is:

For example, if:

```
Q(s,a) = 2
α = 0.1
r = 10
γ = 0.9
max Q(s',a') = 5
```

then:

```
Qnew = 2 + 0.1[10 + (0.9 × 5) - 2]

Qnew = 2 + 0.1[12.5]

Qnew = 3.25
```

This demonstrates how the agent gradually updates its knowledge based on rewards and future expected rewards.

(c) Final Learned Policy

After training, the optimal action for each state is:

```
# Extract final policy
policy = np.argmax(Q, axis=1)

print("\nFinal Learned Policy:")

for row in range(ROWS):

    row_policy = []

    for col in range(COLS):

        state = position_to_state(row, col)

        if state == GOAL:
            row_policy.append("G")

        elif state in OBSTACLES:
            row_policy.append("X")

        else:
            row_policy.append(actions[policy[state]])

    print(row_policy)
```

A typical learned policy may look like:

```
R   R   R   D
D   X   D   D
D   D   X   D
R   R   R   G
```

The exact policy can vary depending on the random seed and training parameters.

The policy should guide the agent:

```
Start
  ↓
Move toward the right/down
  ↓
Avoid obstacles
  ↓
Reach Goal
```

Cumulative Reward Plot

```
plt.figure(figsize=(10, 5))

plt.plot(episode_rewards)

plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Q-Learning: Cumulative Reward per Episode")
plt.grid(True)

plt.show()
```

Convergence Behaviour

Initially, the agent has no knowledge of the environment because all Q-values are zero.

During early episodes:

- Exploration is high.
- The agent takes many incorrect actions.
- It may hit boundaries or obstacles.
- Rewards are relatively low.

As training progresses:

- Successful actions receive higher Q-values.
- The agent learns to avoid obstacles.
- The agent discovers shorter paths to the goal.
- Epsilon decreases.
- The agent increasingly exploits the learned policy.

Therefore, cumulative reward should generally increase and become more stable as the number of episodes increases.

A plateau in cumulative reward indicates that the agent has learned a relatively stable policy.

Overall Analysis of Experiment 2

The Q-Learning experiment demonstrates the complete Reinforcement Learning process:

```
Environment
     ↓
Current State
     ↓
Choose Action
     ↓
Receive Reward
     ↓
Move to Next State
     ↓
Update Q-Value
     ↓
Repeat
```

The agent initially performs random exploration because of the high epsilon value. Through repeated interaction with the Grid World, it gradually learns which actions produce higher long-term rewards.

The discount factor $\gamma = 0.9$ makes future rewards important, allowing the agent to learn paths toward the goal rather than focusing only on immediate rewards.

The learning rate $\alpha = 0.1$ allows gradual Q-value updates, while epsilon decay gradually changes the agent from exploration to exploitation.

Comparison of the Two Experiments

Aspect	Experiment 1	Experiment 2
ML Type	Supervised Learning	Reinforcement Learning
Algorithm	Decision Tree	Q-Learning
Input	Iris features	Grid state
Output	Flower class	Optimal action
Learning	From labelled data	From rewards
Main Concept	Gini Index	Q-value
Decision	Classification	Sequential decision-making
Evaluation	Accuracy, confusion matrix, F1	Cumulative reward, learned policy
Exploration	Not required	Required
Environment interaction	No	Yes